LIV-Surfel: Feed-Forward Surfel Regression for LiDAR-Inertial-Visual Incremental 2DGS Mapping

Puwei Zhao†, Yiduo Zhou†, Yiming Fan, Tong Shi, and Kun Qian*, *Member, IEEE*

***Abstract*—Incremental 3D scene reconstruction requires the system to continuously expand the map while receiving observations frame by frame. In online systems, newly inserted Gaussians undergo only a few optimization steps before they must support subsequent rendering, so their initial attribute quality directly affects the cumulative mapping result. Existing incremental mapping systems based on 3D Gaussian Splatting (3DGS) have two interrelated limitations: 3D ellipsoidal primitives lack explicit surface normal semantics, and rule-based initialization strategies fail to fully exploit external geometric priors. We propose a context-aware feed-forward Gaussian regression system for incremental 2D Gaussian Splatting (2DGS) scene reconstruction. The system adopts 2DGS surfels as the map representation, whose explicit surface orientation allows normal priors to directly constrain primitive orientation. Candidate points are constructed from LiDAR-completed depth and screened by current-map rendering errors. A feed-forward regression network predicts surfel attributes for each candidate point in a single forward pass by aggregating image-rendering context from the current and historical frames via cross-attention, and anchoring surfel rotation on normal priors through Gram-Schmidt orthogonalization. The network is trained through self-distillation without any external annotated dataset. Experiments on the R3LIVE, MCD and our self-collected datasets show that our method outperforms all baselines on nearly all test sequences, achieving an average improvement of 1.28 dB PSNR over the closest multi-sensor incremental Gaussian mapping baseline. Ablation studies further confirm that the feed-forward regressor consistently improves the early convergence quality of new Gaussians under limited optimization budgets. Our code will be released on project page https://github.com/Zhao1231111/LIV-Surfel.**

I. INTRODUCTION

INCREMENTAL 3D scene reconstruction [1]–[4] is essential for time-critical field operations such as disaster response, autonomous driving, and industrial inspection. Recent incremental Gaussian mapping systems [5]–[7] that fuse LiDAR, inertial, and visual measurements have demonstrated online photo-realistic reconstruction in large-scale outdoor environments. These systems use an external front-end to provide stable poses and multi-modal observations [8]–[10], while a Gaussian back-end expands the map frame by frame with Gaussian primitives. However, the per-frame optimization budget of an online system is inherently limited. Newly inserted Gaussians undergo only a few optimization steps before they must serve subsequent rendering, so their initial attribute quality directly affects the cumulative mapping result.

† Puwei Zhao and Yiduo Zhou contributed equally to this work.
∗ Corresponding author: Kun Qian.
Puwei Zhao, Yiduo Zhou, Yiming Fan, Tong Shi, and Kun Qian are with the School of Automation, Southeast University, Nanjing, China (e-mail: kqian@seu.edu.cn).

Existing systems still have interrelated limitations in both map representation and initialization. On the representation side, standard 3DGS [11] uses 3D ellipsoids as primitives that lack explicit surface normal semantics, hindering the direct integration of geometric priors into the map. 2DGS [12] achieves notable geometric improvements by collapsing Gaussians into oriented 2D surfels, yet it has not been widely adopted in incremental mapping. On the initialization side, existing systems predominantly assign attributes to new Gaussians through hand-crafted, rule-based strategies, requiring many optimization steps to converge, which conflicts with the limited per-frame budget. Although some systems introduce depth completion to address sparse LiDAR coverage, learned priors still have not been extended to full attribute prediction.

We propose a context-aware feed-forward Gaussian regression system for incremental 2DGS scene reconstruction. The system adopts 2DGS surfels as the map representation. Given poses from an external LiDAR-Inertial-Visual Odometry (LIVO) front-end, it first leverages a dense depth map obtained by depth completion of sparse LiDAR measurements to generate initial 3D points. To ensure efficient map expansion, the system renders the current map and screens these initial points based on rendering opacity and color errors, extracting a refined set of candidate points that specifically target under-reconstructed regions. For each candidate point, a feed-forward regression network regresses high-quality surfel attributes by aggregating point-aligned multi-view context from the current and historical frames within a sliding window via cross-attention, and anchoring surfel rotations on the geometric normal priors. The network is trained through a self-distillation procedure that requires no external annotated dataset.

The main contributions are as follows:

- A geometry-aware incremental 2DGS mapping back-end. Unlike existing incremental systems that default to volumetric 3D Gaussian primitives, the surface-based 2DGS representation endows the map with explicit normal semantics, enabling the integration of depth completion and normal priors, allowing more geometrically consistent initialization.
- A context-aware feed-forward Gaussian insertion method. Rather than assigning attributes through hand-crafted, rule-based strategies, our network regresses surfel attributes for each candidate point in a single forward pass by aggregating multi-view context via cross-attention and anchoring predictions on normal priors, so that newly inserted Gaussians are closer to their optimized state and suffer less early-stage quality degradation under limited optimization budgets.

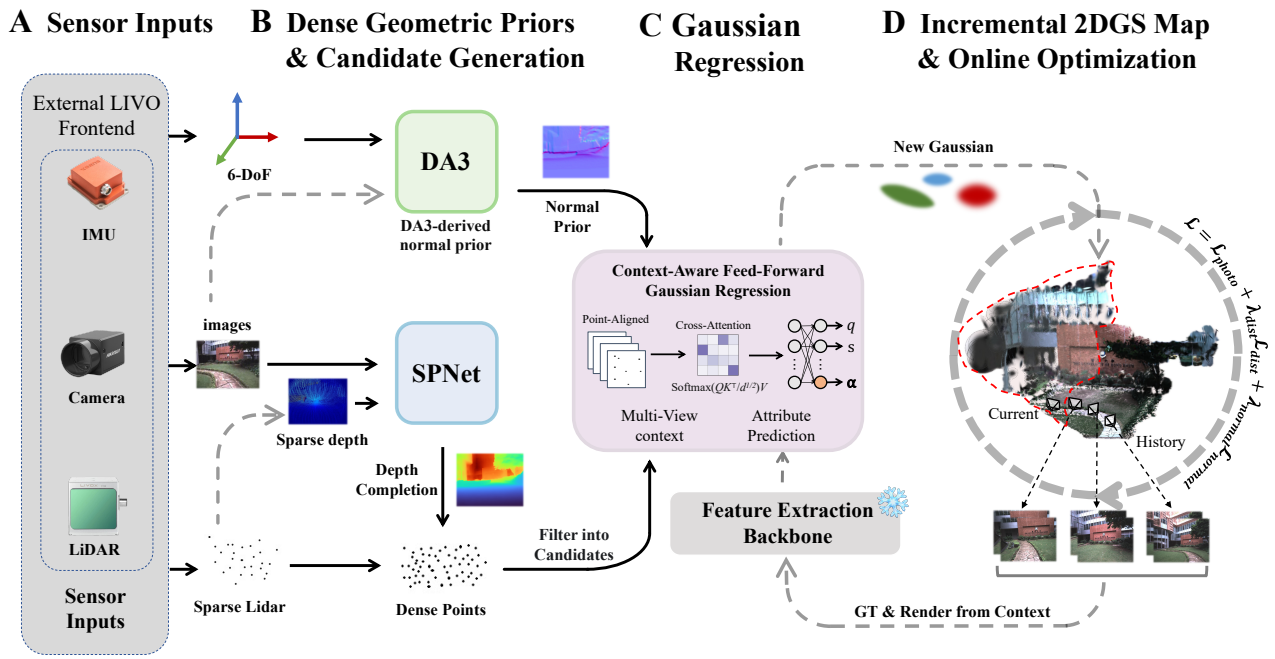

Fig. 1. System pipeline. (A) An external LiDAR-inertial-visual front-end provides synchronized RGB images, LiDAR observations, IMU measurements, and camera poses. (B) The system builds dense geometric priors via SPNet depth completion and DA3 normal estimation, generates initial 3D points from the completed depth map and LiDAR points, and then selects candidate points in under-reconstructed regions based on current-map rendering opacity and color error. (C) For each candidate point, the feed-forward Gaussian regressor aggregates point-aligned multi-view context together with geometric priors and directly predicts 2DGS surfel attributes. (D) The predicted surfels are inserted into the incremental map and further refined by online optimization, while the rendered current map is fed back as context for subsequent insertions.

- A self-distillation training procedure for the feed-forward network. The system collects supervision signals from its own incremental mapping process on training scenes, requiring no external annotated dataset.

II. RELATED WORK

NeRF [13] and its follow-up works have shown the advantages of radiance field representations for novel view synthesis and continuous appearance modeling, yet the costly sampling and integration in volume rendering make it difficult to meet the demands of real-time incremental reconstruction. 3D Gaussian Splatting (3DGS) [11] avoids costly dense ray sampling with explicit Gaussian primitives and differentiable rasterization, achieving clear advantages in rendering speed and optimization efficiency that quickly make it an important foundation for online photo-realistic mapping.

Research around 3DGS advances along two directions. One line of work targets geometric quality: 2DGS [12], GaussianSurfels [14], and LI-GS [15] improve geometric consistency through surface-oriented Gaussians, depth rasterization and normal constraints. These methods suggest that explicit surface-oriented representations are better suited than 3D ellipsoids to incorporate depth and normal priors, a property especially important for incremental systems. Another line of work targets feed-forward Gaussian priors: pixelSplat [16] and MVSplat [17] predict Gaussian attributes from a small number of input images via probabilistic sampling or cost-volume mechanisms. Although these methods primarily target offline multi-view reconstruction from a fixed set of input images, the geometric and appearance priors encoded in their predictions can be exploited for initialization and completion in online systems.

In online mapping, MonoGS [18], GS-SLAM [19], SplaTAM [20], and Photo-SLAM [21] have shown that 3DGS can serve as the unified map representation for online systems, coupling tracking, mapping, and rendering on a single Gaussian map, but these works target only indoor scenes. For large-scale outdoor environments, multi-sensor fusion provides more stable poses and geometric observations. LIV-GaussMap [22] and Gaussian-LIC [23] demonstrate real-time LiDAR-inertial-camera Gaussian mapping, with Gaussian-LIC initializing a 3DGS map from LiDAR points and visual triangulation. GS-LIVM [24] and GS-LIVO [25] further explore outdoor Gaussian mapping under multi-sensor fusion settings, including map expansion, Gaussian optimization, and real-time constraints. Gaussian-LIC2 [26] introduces depth completion priors to improve Gaussian initialization. These works are closest to our problem setting, yet they still primarily adopt a 3DGS back-end, and learned priors are mostly limited to depth completion rather than predicting attributes such as scale, orientation, color, and opacity. We adopt 2DGS surfels as the map representation under LIVO-provided poses and predict surfel attributes through a feed-forward network, thereby improving the initial quality at the incremental insertion stage.

III. METHOD

A. System Overview

The proposed system is an incremental 2DGS scene reconstruction back-end that relies on external pose estimation. It receives a synchronized data stream from an external LiDAR-inertial-camera front-end (e.g., R3LIVE [27]), where each frame contains an RGB image, a sparse LiDAR point cloud, and a 6-DoF camera pose. The system uses 2DGS surfels as the fundamental map representation. The overall pipeline is illustrated in Fig. 1.

When a new keyframe arrives, the system first optimizes its camera pose by rendering the existing Gaussians onto the current frame and minimizing the photometric error. The system then performs map expansion and map optimization as follows. Sparse LiDAR points are projected onto the image plane to form a sparse depth map, which is densified by SPNet [28] depth completion to produce supplementary 3D points in LiDAR-blind regions. Concurrently, the DA3 [29] monocular depth model predicts dense relative depth, which is then aligned to absolute scale via least-squares affine registration with the sparse LiDAR depth, from which per-pixel normals are derived. After the initial 3D points are generated, the system renders the current Gaussian map and identifies under-reconstructed regions based on rendering opacity and color error. Color-gradient filtering and voxel downsampling further select the candidate points that genuinely require new Gaussians. The system then uses these candidates as a mask to filter the feature maps and normal maps, extracting point-aligned features that form the two inputs to the feed-forward regression network: multi-view context features $\mathbf{ctx}$ and geometric priors $\mathbf{g}$. The network aggregates $\mathbf{ctx}$ via cross-attention and regresses surfel attributes for each candidate in a single forward pass. The resulting high-quality new Gaussians are inserted into the map. Online optimization then follows on sampled keyframes, jointly driven by photometric loss $\mathcal{L}_{\text{photo}}$ and geometric constraints ($\mathcal{L}_{\text{dist}}$ and $\mathcal{L}_{\text{normal}}$). Non-keyframes receive only pose updates without triggering Gaussian insertion.

The feed-forward regression network is trained via self-distillation without any external annotated data. The key idea is to construct supervision signals from the system's own online-optimized Gaussian results on training scenes, enabling the network to learn a better surfel initialization strategy.

B. Dense Geometric Priors

We use the lightweight SPNet [28] depth completion network to fuse the RGB image with sparse LiDAR depth into a dense depth map and sample supplementary 3D points in regions lacking LiDAR coverage. The SPNet-complemented points are merged with the original LiDAR projection points to form the initial 3D point set.

Compared with normals obtained by directly differentiating the SPNet-completed depth, the DA3 model [29] has stronger scene-structure priors and produces smoother, more stable surface orientation estimates. We therefore run DA3 on the current RGB frame to obtain a relative depth map, register it to the sparse LiDAR depth via least-squares affine alignment to recover metric scale, and compute per-pixel normals by central differencing. Sampling the normal map at the pixel coordinates of the candidate points yields a camera-space normal $\mathbf{n}_{\text{cam}} \in \mathbb{R}^3$ for each candidate point.

C. Feed-Forward Gaussian Regression Network

As illustrated in Fig. 2, the feed-forward Gaussian regressor operates on the candidate points selected for the current keyframe. Unlike methods such as MVSplat [17] that predict Gaussian parameters densely on an image grid, our setting already provides candidate locations through upstream geometric modules. The network therefore focuses on regressing surfel attributes for these candidates rather than generating Gaussians densely over pixels. Concretely, for each of the N_t candidate points in the current keyframe, we sample and aggregate point-aligned context from multi-view feature maps and learn a mapping from the context features $\mathbf{ctx}$ and geometric priors $\mathbf{g}$ to surfel parameters. Denoting $\mathbf{ctx}_i$ and $\mathbf{g}_i$ as the context and geometric-prior inputs for candidate i, the feed-forward network can be written as:

$$f_{\boldsymbol{\theta}} : \{(\mathbf{ctx}_i, \mathbf{g}_i)\}_{i=1}^{N_t} \mapsto \{(\mathbf{s}_i, \mathbf{q}_i, \alpha_i, \mathbf{c}_i^{\text{dc}}, \mathbf{c}_i^{\text{rest}})\}_{i=1}^{N_t}, \tag{1}$$

where the outputs correspond to the 2D scale, rotation, opacity, and SH color coefficients of each surfel.

1) Context feature input $\mathbf{ctx}$: A shared backbone (following the pixelSplat [16] design, frozen during training) extracts feature maps from the current-frame raw image, the current-frame rendered image, and the raw and rendered images of W historical keyframes within a sliding window. Historical keyframes are introduced because a single viewpoint provides insufficient constraints for surfel attribute regression; rendered images allow the regressor to perceive the current reconstruction state, and their difference from raw images implicitly reflects the degree of under-reconstruction. For each candidate point, the system projects it onto the current frame and each historical keyframe, samples the corresponding feature maps at the projected pixel coordinates using bilinear interpolation, and computes the normalized viewing direction. The raw-image features, rendered-image features, and encoded viewing-direction vectors are concatenated to form the per-point current-frame feature $\mathbf{f}_i^{\text{curr}} \in \mathbb{R}^D$. The same procedure is applied to the W visible historical frames, yielding $\mathbf{f}_i^{\text{hist}} \in \mathbb{R}^{W \times D}$, with a visibility mask $\mathbf{m}_i^{\text{hist}} \in \{0,1\}^W$ marking occluded views. A learnable cross-attention module then compresses the variable-length historical observations into a fixed-dimensional temporal context, where invisible views are excluded by a $-\infty$ mask before the softmax:

$$\mathbf{ctx}_i^{\text{attn}} = \text{softmax}\left(\frac{\mathbf{q}_i \mathbf{K}_i^\top}{\sqrt{d_k}} + \mathbf{M}_i\right)\mathbf{V}_i. \tag{2}$$

The current-frame feature $\mathbf{f}_i^{\text{curr}}$ and the attention output $\mathbf{ctx}_i^{\text{attn}}$ are concatenated to form the encoder input $\hat{\mathbf{ctx}}_i$.

2) Geometric prior input $\mathbf{g}$: The geometric prior is the concatenation of inverse depth, the DA3 camera-space normal, a validity mask, and the KNN pixel spacing:

$$\mathbf{g}_i = [\tilde{z}_i^{-1} \| \mathbf{n}_i^{\text{cam}} \| m_i \| d_i^{\text{nn}}]. \tag{3}$$

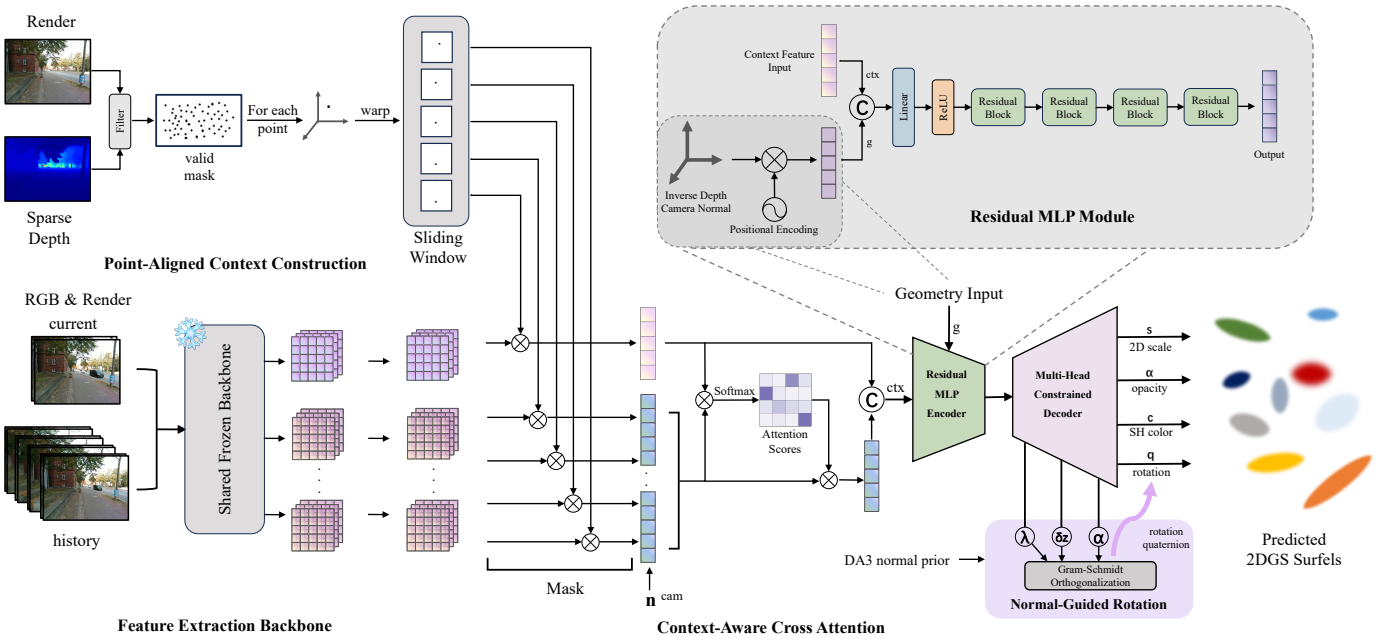

Fig. 2. Architecture of the context-aware feed-forward Gaussian regressor. For the candidate points selected in the current keyframe, the network first constructs point-aligned multi-view context within a sliding window and extracts raw-image and rendered-image features from the current and historical frames through a frozen shared backbone. A cross-attention module then aggregates the historical observations, and the resulting context features are fused with geometric priors including inverse depth, DA3 normals, validity masks, and positional encodings in a residual MLP encoder. Finally, a multi-head constrained decoder predicts 2DGS surfel attributes, including scale, rotation, opacity, and SH color coefficients; the normal-guided rotation branch constructs a geometrically consistent orientation via Gram-Schmidt orthogonalization.

Here $\bar{z}_i^{-1}$ provides the depth information needed for scale recovery, $\mathbf{n}_i^{\text{cam}}$ serves as the normal anchor for surfel orientation, m_i flags whether the geometric prior is valid, and d_i^{nn} captures the local sampling density. Each continuous component undergoes NeRF-style positional encoding and is concatenated with $\hat{\mathbf{ctx}}_i$.

3) Encoder and Decoder: The encoder consists of one linear projection followed by four pre-activation residual blocks and outputs a 512-dimensional latent vector $\mathbf{h}_i$. The decoder consists of multiple prediction heads which are each implemented as a Linear $\rightarrow$ ReLU $\rightarrow$ Linear structure, to regress different attributes. Each head output is not used directly as the final Gaussian parameter but is decoded through upstream geometric priors, anchoring predictions on existing geometric information for greater stability.

a) Normal-guided rotation decoding. Rather than regressing a quaternion directly, the system takes the DA3 camera-space normal $\mathbf{n}_i^{\text{cam}}$ as an anchor and lets the network predict a correction. The network outputs a normal correction vector $\delta\mathbf{z}_i \in \mathbb{R}^3$, a blending strength $\lambda_i = \text{softplus}(\cdot) \in \mathbb{R}_{>0}$, and an auxiliary vector $\mathbf{a}_i \in \mathbb{R}^3$, from which the corrected normal direction is obtained:

$$\mathbf{z}_i = \text{normalize}(\mathbf{n}_i^{\text{cam}} + \lambda_i \cdot \delta\mathbf{z}_i). \tag{4}$$

A full orthonormal basis is then constructed via Gram-Schmidt orthogonalization:

$$\mathbf{y}_i = \text{normalize}(\mathbf{z}_i \times \mathbf{a}_i), \qquad \mathbf{x}_i = \mathbf{y}_i \times \mathbf{z}_i. \tag{5}$$

The camera-frame rotation matrix $[\mathbf{x}_i \mid \mathbf{y}_i \mid \mathbf{z}_i]$ is transformed to the world frame through the current pose R_{wc} and stored as a unit quaternion.

b) Density-adaptive scale decoding. The scale head predicts the relative surfel size in the pixel plane, modulated by the local point-density prior:

$$\mathbf{s}_i^{\text{pixel}} = \frac{d_i^{\text{nn}}}{2} \cdot \exp(\text{head}(\mathbf{h}_i)) \in \mathbb{R}^2_{>0}, \tag{6}$$

where d_i^{nn} is the KNN pixel spacing of the candidate point. The world-space scale is recovered by

$$\mathbf{s}_i^{\text{world}} = \mathbf{s}_i^{\text{pixel}} / (\bar{z}_i^{-1} \cdot f). \tag{7}$$

This parameterization avoids requiring the network to learn depth-dependent absolute scales and instead only learns relative scale variations related to local density.

c) Color decoding. The DC color head outputs a residual $\Delta\mathbf{c}_i^{\text{dc}}$ that is added to a base SH prior converted from the single-frame RGB observation:

$$\mathbf{c}_i^{\text{dc}} = \text{RGB2SH}(\mathbf{I}_i) + \Delta\mathbf{c}_i^{\text{dc}}. \tag{8}$$

Higher-order SH coefficients are predicted by a separate head. The residual design lets the network start from a reasonable color prior and learn only local corrections.

d) Opacity decoding. The opacity head applies a sigmoid activation to produce $\alpha_i \in (0, 1)$, which is then converted to logit space for storage.

D. Self-Distillation Training

The feed-forward network is trained without any external multi-view annotated dataset through the following self-distillation pipeline.

1) Online data collection. The system first runs a complete incremental mapping pass on training scenes in a naive mode

TABLE I
QUANTITATIVE RENDERING PERFORMANCE ON PUBLIC MULTIMODAL AND SELF-COLLECTED CAMPUS DATASETS. FORMAT: PSNR↑ | SSIM↑ | LPIPS↓. BEST RESULTS ARE SHOWN IN BOLD.

Sequence	Rendering Quality (PSNR↑ | SSIM↑ | LPIPS↓)			
	MonoGS [18]	MM3DGS-SLAM [30]	Gaussian-LIC2 [26]	Ours
R3LIVE (Avia)				
degenerate_seq_00	14.59 | 0.460 | 0.624	19.35 | 0.698 | 0.212	21.99 | 0.782 | 0.162	**23.41** | **0.821** | **0.137**
degenerate_seq_01	14.66 | 0.438 | 0.692	18.57 | 0.621 | 0.277	22.94 | 0.790 | 0.159	**23.54** | **0.810** | **0.157**
hku_campus_00	15.41 | 0.464 | 0.675	18.70 | 0.621 | 0.319	24.34 | 0.787 | 0.156	**25.52** | **0.820** | **0.152**
hku_campus_01	8.39 | 0.065 | 0.873	16.97 | 0.593 | 0.271	20.52 | 0.674 | 0.236	**23.15** | **0.754** | **0.174**
hku_park_00	13.24 | 0.319 | 0.733	14.98 | 0.363 | 0.396	17.17 | 0.469 | 0.340	**17.80** | **0.491** | **0.331**
hku_park_01	12.22 | 0.289 | 0.790	15.79 | 0.392 | 0.409	19.46 | 0.529 | 0.325	**20.65** | **0.580** | **0.248**
MCD (OS1-64)				
tuhh_day_02	8.72 | 0.158 | 0.893	11.15 | 0.478 | 0.350	20.35 | 0.626 | 0.262	**20.99** | **0.652** | **0.249**
tuhh_day_03	8.09 | 0.136 | 0.896	11.36 | 0.542 | 0.301	21.38 | 0.672 | **0.229**	**21.82** | **0.688** | 0.238
tuhh_day_04	11.73 | 0.250 | 0.805	13.86 | 0.384 | 0.398	**19.27** | **0.528** | 0.310	19.07 | 0.506 | **0.309**
Self-Collected Dataset (Avia)				
seu_campus_seq_00	14.56 | 0.411 | 0.732	18.54 | 0.563 | 0.366	23.97 | 0.769 | 0.215	**25.55** | **0.830** | **0.145**
seu_campus_seq_01	13.21 | 0.392 | 0.796	17.14 | 0.528 | 0.421	22.76 | 0.711 | 0.195	**24.56** | **0.797** | **0.161**

that does not use the feed-forward network but instead applies rule-based initialization. During this pass, when the cumulative training count of a frame's newly added Gaussians reaches a preset threshold, the system snapshots their current attributes as supervision signals and simultaneously projects the current Gaussian map into into each frame of the current sliding window to construct the corresponding context features $\mathbf{ctx}_i$ and geometric priors $\mathbf{g}_i$ following the definitions above. Each training sample is thus organized as the input-output pair $(\mathbf{ctx}_i, \mathbf{g}_i) \mapsto (\mathbf{s}_i, \mathbf{q}_i, \alpha_i, \mathbf{c}_i^{\mathrm{dc}}, \mathbf{c}_i^{\mathrm{rest}})$.

2) Scale-rotation ambiguity resolution. A 2DGS surfel has an inherent scale-rotation ambiguity: swapping the two tangent-direction scales and rotating by 90° yields an equivalent representation. If unresolved, the same geometry maps to two different parameter combinations in the training labels, destabilizing learning. During data preparation, this ambiguity is explicitly resolved: for each ground-truth Gaussian, $s_1 \geq s_2$ is enforced; when a swap is needed, the quaternion is simultaneously rotated by 90°:

$$\mathbf{q}' = \mathrm{normalize}\left(\frac{(w-z,\ x+y,\ y-x,\ w+z)}{\sqrt{2}}\right). \tag{9}$$

3) Training objective. The network is trained with a multi-task loss covering all predicted attributes. Scale loss is measured by L1 distance in log world-scale space. Rotation loss uses the quaternion cosine distance $\mathcal{L}_{\mathrm{rot}} = 1-|\langle \mathbf{q}_{\mathrm{pred}}, \mathbf{q}_{\mathrm{gt}} \rangle|$. Color losses supervise DC and higher-order SH coefficients separately with MSE. Opacity loss is an L1 distance.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

We validate the proposed method from two perspectives: (i) quantitative and qualitative rendering-quality comparisons with existing incremental Gaussian mapping methods on the public R3LIVE [27] (Avia) dataset, the MCD [31] (OS1-64) dataset, and our self-collected dataset captured on the Southeast University Sipailou campus (using a Livox Avia LiDAR, a Hikvision MV-CA013-21UC RGB camera, and the 200 Hz IMU integrated in the LiDAR); (ii) ablation analysis of the feed-forward regressor on *hku_campus_seq_00*, focusing on its effect on new-Gaussian initialization quality and early convergence under limited optimization budgets. Rendering quality is measured by PSNR, SSIM, and LPIPS. All methods are evaluated under the same data splits, with non-training frames used for within-sequence novel-view evaluation. To ensure a fair comparison, the total optimization steps for every method on each sequence are capped at 20 times the number of image frames.

All experiments are conducted on a unified hardware and software platform: Ubuntu 20.04 (Python 3.8), PyTorch 2.0.0, CUDA 11.8, a single RTX 4090D (24 GB) GPU, a 16-vCPU Intel Xeon Platinum 8481C processor, and 80 GB system memory.

The feed-forward regression network is trained on R3LIVE sequences *hku_campus_seq_02*, *hku_campus_seq_03*, *hkust_campus_seq_02*, *hkust_campus_seq_03*, and our self-collected sequences *seu_campus_seq_02*, *seu_campus_seq_03*. Training uses the Adam optimizer with a point-level batch size of 8192 and gradient clipping. For network training, we set the learning rate to 5×10^{-4}, the number of epochs to 150, λ_{scale} to 1.0, λ_{rot} to 1.0, λ_{dc} to 1.0, λ_{rest} to 0.1, λ_{opac} to 1.0, and the context window size to 15. We use Coco-LIC [10] as the external LiDAR-inertial-camera front-end of our system.

Baselines include MonoGS [18], MM3DGS-SLAM [30], and Gaussian-LIC2 [26]. Baseline statistics on R3LIVE and MCD are taken from the Gaussian-LIC2 paper [26]. On our self-collected dataset and for qualitative renderings, MonoGS

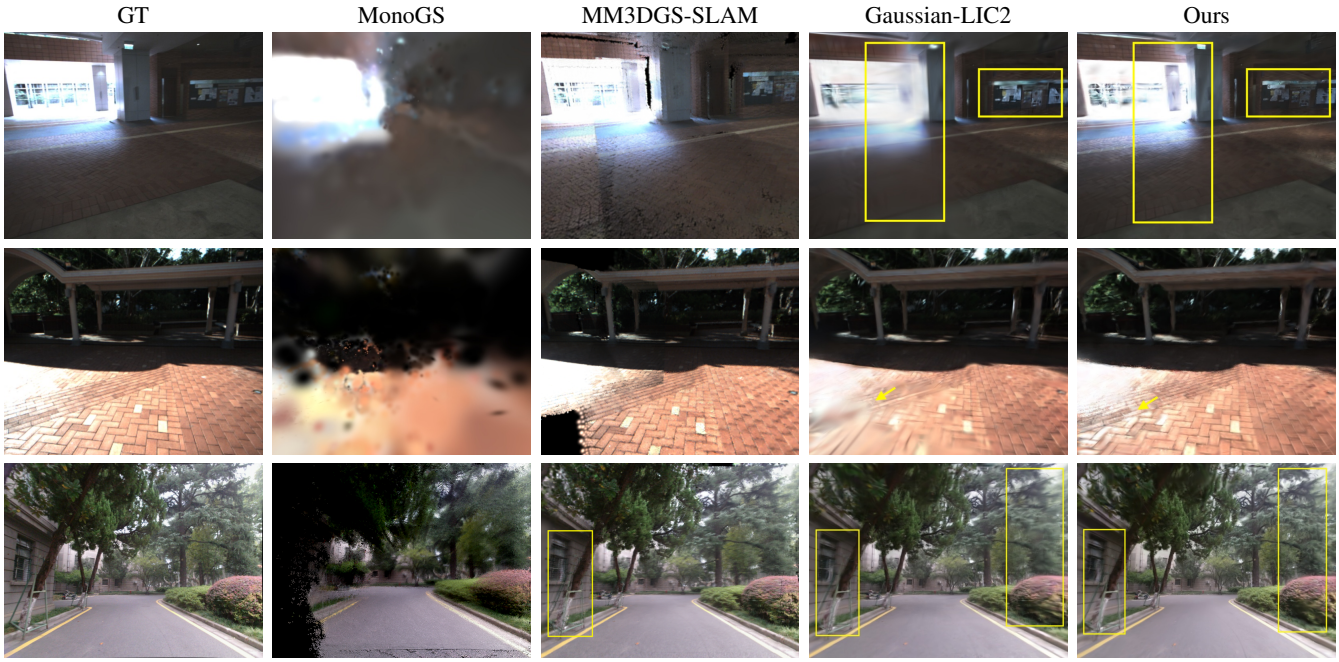

Fig. 3. Qualitative comparison on public and self-collected datasets. Row 1: *hku_campus_00* frame 690. Row 2: *degenerate_seq_00* frame 516. Row 3: *seu_campus_seq_01* frame 1080. Compared with MonoGS, MM3DGS-SLAM, and Gaussian-LIC2, our method preserves sharper details at structural boundaries, ground textures, and distant regions, while significantly reducing blur, trailing artifacts, and local distortion.

runs in vision-only mode; MM3DGS-SLAM runs in RGB-D mode with pseudo RGB-D data formed by projecting LiDAR points onto images; Gaussian-LIC2 uses the February 2026 open-source release.

B. Comparison with Existing Methods

Table I presents the quantitative results on R3LIVE (Avia), MCD (OS1-64), and our self-collected dataset (Avia). Our method achieves the best rendering metrics on nearly all test sequences. Vision-only MonoGS shows noticeable blur, floaters, and structural collapse in large-scale outdoor and degenerate scenes, indicating that relying solely on subsequent optimization to correct inaccurate initialization is difficult. MM3DGS-SLAM, using pseudo RGB-D data, produces better results but its isotropic and view-independent Gaussian distribution underperforms under limited optimization steps. With the SPNet point-augmentation strategy, Gaussian-LIC2 successfully reconstructs regions never scanned by the LiDAR and achieves clear overall improvements, yet its new Gaussian attributes still rely on hand-crafted rules, resulting in trailing artifacts, over-smoothing, or local distortion in highlight regions, boundary areas, and under-observed zones under limited optimization. In contrast, our method uses image-rendering context from the current and historical frames to provide surfel attribute initialization closer to convergence, yielding more stable appearance and sharper structures after only a few optimization steps.

C. Ablation Study: Effect of the Feed-Forward Regressor

To verify the contribution of the feed-forward regressor to incremental 2DGS mapping, we conduct an ablation study on *hku_campus_00*. We denote the system without the regressor (using rule-based initialization) as Baseline and the full system as Ours. Unlike comparisons on a few representative frames, we compute statistics from the optimization trajectories of all training frames over the steps after insertion. The evaluation focuses on two aspects: (i) rendering quality at 16 steps after insertion and (ii) the number of steps after insertion required to reach preset metric-specific thresholds. The PSNR, SSIM, and LPIPS thresholds are set to 25.0, 0.70, and 0.18, respectively.

TABLE II

ABLATION RESULTS ON *hku_campus_00*. STATISTICS ARE COMPUTED OVER THE OPTIMIZATION TRAJECTORIES OF ALL TRAINING FRAMES. PART (A) REPORTS RENDERING QUALITY AT 16 STEPS AFTER INSERTION; PART (B) REPORTS THE MEAN ± STD STEPS AFTER INSERTION REQUIRED TO REACH PRESET QUALITY THRESHOLDS.

(a) Rendering quality at 16 steps after insertion

Setting	PSNR↑	SSIM↑	LPIPS↓
Baseline	23.97 ± 2.91	0.800 ± 0.071	0.213 ± 0.055
Ours	**24.34 ± 2.74**	**0.813 ± 0.063**	**0.195 ± 0.048**

(b) Steps after insertion to reach preset quality thresholds

Setting	PSNR $\geq$ 25.0 ↓	SSIM $\geq$ 0.70 ↓	LPIPS $\leq$ 0.18 ↓
Baseline	20.4 ± 28.1	3.2 ± 6.2	27.0 ± 24.5
Ours	**17.7 ± 26.0**	**3.1 ± 10.2**	**19.6 ± 15.6**

Table II shows that introducing the feed-forward regressor consistently improves early convergence across training frames. Under a fixed budget of 16 steps after insertion, Ours outperforms Baseline on all three rendering metrics. Ours also reaches the preset metric-specific thresholds in fewer average steps under all three thresholds. Table III further lists the mean

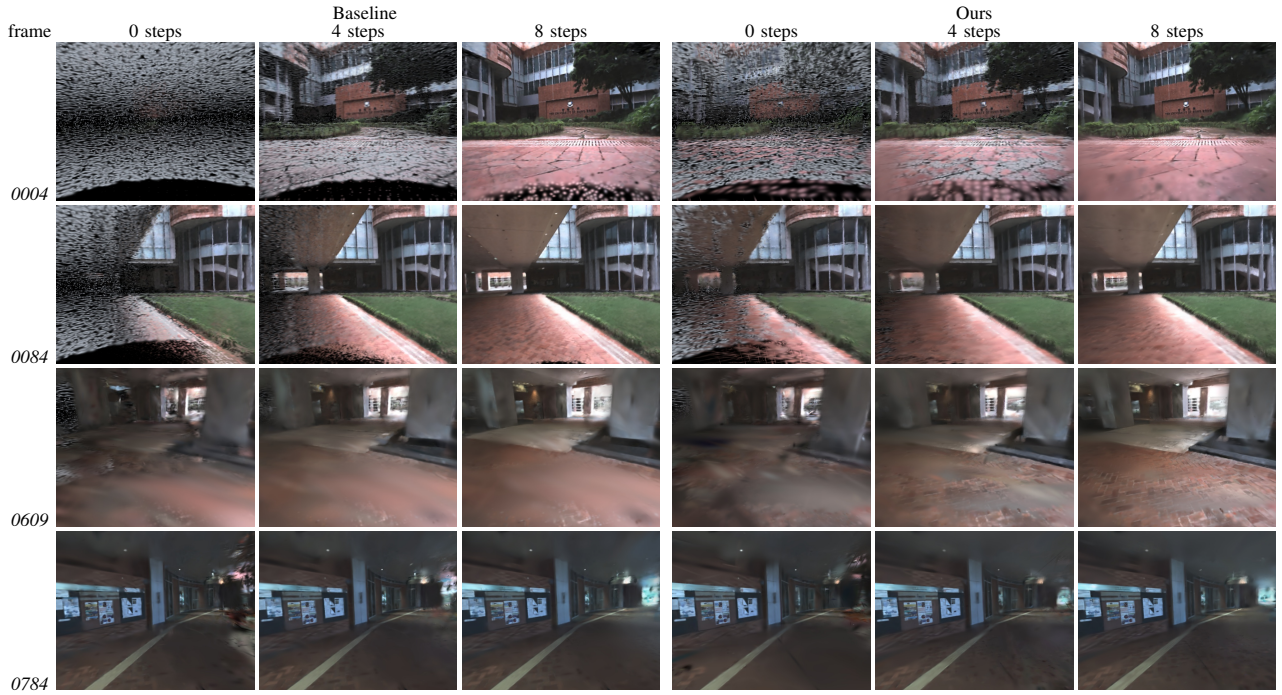

Fig. 4. Ablation visualization of the feed-forward regressor. Representative frames *train_0004*, *train_0084*, *train_0609*, and *train_0784* are shown after 0, 4, and 8 post-insertion optimization steps. Left three columns: Baseline; right three columns: Ours.

convergence trends of the three metrics over the first 16 steps after insertion.

The visualization results in Fig. 4 intuitively explain the improved early convergence. Without the regressor, initial Gaussian parameters are given by hand-crafted rules and show strong randomness and disorder, requiring multiple optimization rounds to align with the scene. With the regressor, the scene layout and dominant structures are already recognizable at step 0. The regressor shows adaptive tendencies across different regions: in under-reconstructed or poorly covered areas, the predicted Gaussians tend toward larger coverage to quickly fill in the dominant structure; in regions with adequate geometric support or rich detail, the predictions tend toward finer-grained Gaussians to avoid over-covering details. Similarly, in low-texture or large smooth areas, the regressor tends to produce larger Gaussians for efficient early coverage, whereas in regions with richer texture and denser structural boundaries, the predicted Gaussians are typically smaller and more concentrated to better capture fine detail. As optimization proceeds, our method converges to clear and stable renderings more rapidly.

The experiments lead to two conclusions. First, in the full novel-view rendering evaluation, our method outperforms all baselines on nearly all test sequences, showing that combining 2DGS representation with context-aware feed-forward insertion consistently improves the rendering quality of incremental Gaussian maps. Second, the core value of the feed-forward regressor lies not merely in slightly higher final metrics but in the improvement of the initial quality and early convergence speed after new Gaussian insertion. This property is important for online systems, because better initial Gaussians not only reduce local artifacts but also provide more reliable rendering supervision for subsequent frames, creating a positive feedback loop that continuously improves the overall map quality.

TABLE III
ABLATION STUDY ON EARLY CONVERGENCE OF THE FEED-FORWARD REGRESSOR ON *hku_campus_00*. VALUES ARE AVERAGED OVER VALID TRAINING FRAMES; BOLD INDICATES THE BETTER RESULT AT EACH STEP.

Metric	Method	Steps after insertion				
		0	**4**	**8**	**12**	**16**
PSNR↑	Baseline	18.18	22.07	23.41	23.95	23.97
	Ours	**18.92**	**22.83**	**23.83**	**24.20**	**24.34**
SSIM↑	Baseline	0.636	0.753	0.786	0.800	0.800
	Ours	**0.661**	**0.772**	**0.797**	**0.810**	**0.813**
LPIPS↓	Baseline	0.380	0.291	0.248	0.226	0.213
	Ours	**0.366**	**0.276**	**0.233**	**0.211**	**0.195**

V. CONCLUSION

We address the problem that the initial quality of newly inserted Gaussians determines the cumulative mapping result in incremental scene reconstruction and propose a context-aware feed-forward Gaussian regression system based on 2DGS surfels. The method fuses contextual information with geometric priors to directly predict new surfel attributes at the insertion stage, replacing traditional rule-based initialization. The network is trained through self-distillation without additional annotated data.

Experiments on R3LIVE and MCD show that the proposed method consistently improves novel-view rendering quality of incremental Gaussian maps, achieving an average improve-

ment of 1.28 dB PSNR on R3LIVE over the closest multi-sensor baseline. Ablation studies further show that the main benefit comes from improved initial surfel state and early convergence quality, which yields superior mapping quality within a limited number of optimization steps. The current method still relies on a stable external front-end; future work will explore incremental Gaussian mapping under weaker prior conditions.

REFERENCES

[1] F. Tosi, Y. Zhang, Z. Gong, E. Sandström, S. Mattoccia, M. R. Oswald, and M. Poggi, "How NeRFs and 3-D Gaussian Splatting are reshaping SLAM: A survey," *IEEE Transactions on Robotics*, vol. 42, pp. 1405–1427, 2026.

[2] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "iMAP: Implicit mapping and positioning in real-time," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 6209–6218.

[3] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, "NICE-SLAM: Neural implicit scalable encoding for SLAM," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 776–12 786.

[4] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint coordinate and sparse parametric encodings for neural real-time SLAM," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 13 293–13 302.

[5] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, "Gaussian-SLAM: Photo-realistic dense SLAM with Gaussian Splatting," *arXiv preprint arXiv:2312.10070*, 2023.

[6] C. Wu, Y. Duan, X. Zhang, Y. Sheng, J. Ji, and Y. Zhang, "MM-Gaussian: 3D Gaussian-based multi-modal fusion for localization and reconstruction in unbounded scenes," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 12 287–12 293.

[7] C. Zhao, S. Sun, R. Wang, Y. Guo, J.-J. Wan, Z. Huang, X. Huang, Y. V. Chen, and L. Ren, "TCLC-GS: Tightly coupled LiDAR-Camera Gaussian Splatting for autonomous driving," *arXiv preprint arXiv:2404.02410*, 2024.

[8] J. Lin and F. Zhang, "R^{3}LIVE++: A Robust, Real-Time, Radiance Reconstruction Package With a Tightly-Coupled LiDAR-Inertial-Visual State Estimator," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 11 168–11 185, 2024.

[9] C. Zheng, W. Xu, Z. Zou, T. Hua, C. Yuan, D. He, B. Zhou, Z. Liu, J. Lin, F. Zhu, Y. Ren, R. Wang, F. Meng, and F. Zhang, "FAST-LIVO2: Fast, direct LiDAR-Inertial-Visual odometry," *IEEE Transactions on Robotics*, vol. 41, pp. 326–346, 2025.

[10] X. Lang, C. Chen, K. Tang, Y. Ma, J. Lv, Y. Liu, and X. Zuo, "Coco-LIC: Continuous-time tightly-coupled LiDAR-Inertial-Camera odometry using non-uniform B-Spline," *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 7074–7081, 2023.

[11] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, 2023.

[12] B. Huang, Z. Yu, A. Chen, A. Geiger, and S. Gao, "2D Gaussian Splatting for geometrically accurate radiance fields," in *ACM SIGGRAPH 2024 Conference Papers*, 2024, pp. 1–11.

[13] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," in *European Conference on Computer Vision*. Springer, 2020, pp. 405–421.

[14] P. Dai, J. Xu, W. Xie, X. Liu, H. Wang, and W. Xu, "High-quality surface reconstruction using Gaussian surfels," in *ACM SIGGRAPH 2024 Conference Papers*, 2024, pp. 1–11.

[15] C. Jiang, R. Gao, K. Shao, Y. Wang, R. Xiong, and Y. Zhang, "LI-GS: Gaussian Splatting with LiDAR incorporated for accurate large-scale reconstruction," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1864–1871, 2025.

[16] D. Charatan, S. L. Li, A. Tagliasacchi, and V. Sitzmann, "pixelSplat: 3D Gaussian splats from image pairs for scalable generalizable 3D reconstruction," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 19 457–19 467.

[17] Y. Chen, H. Xu, C. Zheng, B. Zhuang, M. Pollefeys, A. Geiger, T.-J. Cham, and J. Cai, "MVSplat: Efficient 3D Gaussian Splatting from sparse multi-view images," in *European Conference on Computer Vision*. Springer, 2024, pp. 370–386.

[18] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, "Gaussian Splatting SLAM," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 18 039–18 048.

[19] C. Yan, D. Qu, D. Xu, B. Zhao, Z. Wang, D. Wang, and X. Li, "GS-SLAM: Dense visual SLAM with 3D Gaussian Splatting," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 19 595–19 604.

[20] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, "SplaTAM: Splat, track & map 3D gaussians for dense RGB-D SLAM," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 21 357–21 366.

[21] H. Huang, L. Li, H. Cheng, and S.-K. Yeung, "Photo-SLAM: Real-time simultaneous localization and photorealistic mapping for monocular, stereo, and RGB-D cameras," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 21 584–21 593.

[22] S. Hong, J. He, X. Zheng, and C. Zheng, "LIV-GaussMap: LiDAR-Inertial-Visual fusion for real-time 3D radiance field map rendering," *IEEE Robotics and Automation Letters*, vol. 9, no. 11, pp. 9765–9772, 2024.

[23] X. Lang, L. Li, C. Wu, C. Zhao, L. Liu, Y. Liu, J. Lv, and X. Zuo, "Gaussian-LIC: Real-time photo-realistic SLAM with Gaussian Splatting and LiDAR-Inertial-Camera fusion," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 8500–8507.

[24] Y. Xie, Z. Huang, J. Wu, and J. Ma, "GS-LIVM: Real-time photo-realistic LiDAR-Inertial-Visual mapping with Gaussian Splatting," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 26 869–26 878.

[25] S. Hong, C. Zheng, Y. Shen, C. Li, F. Zhang, T. Qin, and S. Shen, "GS-LIVO: Real-time LiDAR, inertial, and visual multisensor fused odometry with Gaussian mapping," *IEEE Transactions on Robotics*, vol. 41, pp. 4253–4268, 2025.

[26] X. Lang, J. Lv, K. Tang, L. Li, J. Huang, L. Liu, Y. Liu, and X. Zuo, "Gaussian-LIC2: LiDAR-Inertial-Camera Gaussian Splatting SLAM," *arXiv preprint arXiv:2507.04004*, 2025.

[27] J. Lin and F. Zhang, "R^{3}LIVE: A robust, real-time, RGB-colored, LiDAR-Inertial-Visual tightly-coupled state estimation and mapping package," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 10 672–10 678.

[28] T. Li, S. Luo, Z. Fan, Q. Zhou, and T. Hu, "SPNet: Structure preserving network for depth completion," *PLOS ONE*, vol. 18, no. 1, p. e0280886, 2023.

[29] H. Lin, S. Chen, J. Liew, D. Y. Chen, Z. Li, G. Shi, J. Feng, and B. Kang, "Depth Anything 3: Recovering the visual space from any views," *arXiv preprint arXiv:2511.10647*, 2025.

[30] L. C. Sun, N. P. Bhatt, J. C. Liu, Z. Fan, Z. Wang, T. E. Humphreys, and U. Topcu, "MM3DGS SLAM: Multi-modal 3D Gaussian Splatting for SLAM using vision, depth, and inertial measurements," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 10 159–10 166.

[31] T.-M. Nguyen, S. Yuan, T. H. Nguyen, P. Yin, H. Cao, L. Xie, M. Wozniak, P. Jensfelt, M. Thiel, J. Ziegenbein, and N. Blunder, "MCD: Diverse large-scale multi-campus dataset for robot perception," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 22 304–22 313.